GRUPO 13 / ARQUITECTURA DE SISTEMAS OPERATIVOS

Implementación de una Máquina Virtual Simple

Implementación de una máquina virtual simple

Institución	[Completar institución]
Asignatura	Arquitectura de Sistemas Operativos
Proyecto	Implementación de una máquina virtual simple
Grupo	13
Integrantes	[Completar integrantes]
Docente	[Completar docente]
Fecha de entrega	[Completar fecha de entrega]
Fecha de revisión	26 de octubre de 2026
Lenguaje de implementación	Python 3
Producto	Simulador con memoria, registros, unidad de control, ciclo Fetch-Decode-Execute, dispositivos virtuales y pruebas reproducibles.

Este documento reúne el diseño y los casos de prueba del Entregable 1. El paquete incluye manual de usuario, manual técnico, evidencias de ejecución y guía de defensa para el Entregable 2. Los campos entre corchetes requieren datos reales del grupo antes de la impresión definitiva. La fecha de revisión identifica esta versión y no sustituye la fecha de entrega.

La propuesta menciona Python entre las opciones de implementación y también contiene indicaciones específicas de Java. El proyecto conserva Python según autorización consignada por el grupo. Este documento registra esa decisión; no constituye una verificación externa de la autorización docente.

Introducción

Una máquina virtual permite estudiar el comportamiento de un procesador mediante un programa que representa su memoria, registros y reglas de ejecución. En este proyecto se construye una arquitectura didáctica con 100 celdas de memoria y un lenguaje reducido de diez instrucciones. La máquina carga programas de texto, interpreta sus instrucciones y muestra los cambios que producen.

El simulador ofrece una interfaz gráfica con Tkinter, una interfaz de consola y un modo de ejecución de archivos sin diálogos. Utilizan el mismo motor. El usuario puede ejecutar una instrucción completa por paso, observar las fases Fetch, Decode y Execute, revisar registros y memoria, y comprobar operaciones de entrada y salida. La ejecución incorpora un límite total de ciclos desde carga o reinicio para detectar programas que no alcanzan su instrucción de terminación. Los reportes HTML, CSV y JSON conservan los resultados y el historial de la ejecución.

Planteamiento del problema

La ejecución de instrucciones ocurre a un nivel que normalmente no es visible al usar una computadora. Esto dificulta relacionar conceptos como contador de programa, acumulador, memoria y unidad de control con una secuencia concreta de operaciones.

El problema que resuelve el proyecto es representar esas operaciones en una herramienta que permita cargar un programa pequeño, seguirlo instrucción por instrucción y justificar su resultado. La representación debe distinguir instrucciones y datos, controlar accesos a memoria, informar errores y permitir repetir una prueba con las mismas condiciones iniciales.

Objetivos

Objetivo general

Desarrollar una máquina virtual simple que interprete un lenguaje de instrucciones y muestre el funcionamiento coordinado de memoria, registros, unidad de control y dispositivos virtuales mediante el ciclo Fetch-Decode-Execute.

Objetivos específicos

- Representar una memoria de 100 celdas para instrucciones y datos enteros.
- Implementar los registros PC, IR y ACC, y las banderas de estado Z y N.
- Ejecutar instrucciones aritméticas, de almacenamiento, salto, entrada, salida y terminación.
- Cargar programas desde archivos `.asm` o desde el editor y validar su sintaxis antes de modificar la máquina.
- Mostrar la instrucción ejecutada, las fases del ciclo, las modificaciones de memoria y los resultados de salida.
- Permitir ejecución por pasos, ejecución automática, pausa y reinicio en la interfaz gráfica.
- Detectar instrucciones inexistentes, direcciones inválidas, datos incompatibles y límites de ciclos.
- Validar el motor y sus interfaces mediante programas de prueba y evidencias reproducibles.
- Generar reportes con código, registros, memoria, entradas, salidas, cambios por instrucción y diagnóstico de errores.

Justificación

El simulador permite observar cómo una instrucción guardada en memoria produce un cambio concreto en el estado de una máquina. Por ejemplo, `LOAD` copia un dato al acumulador, `ADD` modifica ese acumulador y `STORE` escribe el resultado en memoria. Los saltos muestran que el orden de ejecución depende del contador de programa y del estado de las banderas.

La separación entre el motor y las interfaces facilita estudiar los componentes y ejecutar las mismas pruebas por distintos medios. Los datos declarados con `DATA`, los programas de ejemplo y las evidencias exportadas permiten reproducir resultados y explicar errores durante la defensa del proyecto.

Marco teórico

Máquina virtual: representación por software de una arquitectura con reglas propias para almacenar y ejecutar instrucciones. La máquina de este proyecto tiene un propósito educativo y no ejecuta programas del procesador físico de la computadora.

Memoria principal: conjunto de celdas identificadas por direcciones. La máquina utiliza 100 celdas, de 0 a 99. Una celda puede contener una instrucción o un entero; el acceso numérico a una celda que contiene código produce un error.

Contador de programa, PC: dirección de la próxima instrucción. En una instrucción normal avanza una posición. Un salto puede reemplazar ese valor por la dirección de destino.

Registro de instrucción, IR: conserva la instrucción obtenida de memoria durante Fetch. Permite identificar la operación que se está procesando o que acaba de completarse.

Acumulador, ACC: registro de trabajo para cargar datos, realizar sumas y restas, recibir entrada y producir salida.

Unidad de control: lógica que obtiene la instrucción, identifica su operación y dirige la ejecución. En esta implementación se encuentra principalmente en `VirtualMachine.step()` y `VirtualMachine._execute()`.

Banderas, FLAGS: valores que describen el acumulador. `Z` vale 1 si `ACC = 0`; `N` vale 1 si `ACC < 0`. Se actualizan después de cada instrucción completada. Al iniciar, `ACC = 0`, por lo que `Z = 1` y `N = 0`.

Ciclo Fetch-Decode-Execute: secuencia que obtiene una instrucción de memoria, interpreta su operación y ejecuta su efecto. Un paso de la aplicación completa las tres fases; las tarjetas muestran las fases registradas, sin detenerse entre ellas.

Controladores de dispositivos: adaptadores para entrada por teclado, lectura de archivos y salida a una impresora virtual. La impresora es un registro de valores emitidos por `OUTPUT`; no necesita una impresora física.

Diseño de la arquitectura

El sistema utiliza los siguientes componentes:

Componente	Representación	Responsabilidad
Memoria	Clase `Memory`, 100 celdas	Guardar instrucciones y datos; validar direcciones y lecturas numéricas.
Instrucción	Clase `Instruction`	Conservar opcode y operando opcional.
Programa cargado	Clase `Program`	Agrupar instrucciones y datos declarados con DATA.
Registros y estado	Atributos de `VirtualMachine`	Mantener PC, IR, ACC, Z, N, detención, error y ciclos.
Unidad de control	`step()` y `_execute()`	Coordinar Fetch, Decode y Execute y aplicar cada opcode.
Teclado	`KeyboardController`	Obtener un entero para INPUT.
Disco	`DiskController`	Leer y analizar archivos ASM.
Impresora	`PrinterController`	Mostrar ACC al ejecutar OUTPUT.

Componente	Representación	Responsabilidad
Interfaces	`run_gui()` y `run_cli()`	Cargar programas, ejecutarlos y revisar el estado.

Los controladores específicos utilizan las funciones compartidas de `DeviceControllers`. El motor acepta funciones de entrada y salida para poder probarlo sin diálogos. La interfaz gráfica añade editor, filtros de memoria, planificación de la ejecución y reportes. La implementación se organiza por responsabilidad: `motor.py` contiene la arquitectura y los controladores, `interfaz.py` construye el escritorio, `consola.py` ofrece menú y argumentos, y `reportes.py` crea y guarda las instantáneas. `simulador_vm.py` conserva el acceso público compatible.

`main(argv=None)` selecciona el modo. Sin argumentos abre la interfaz; `--cli` abre el menú; `--archivo RUTA` ejecuta un ASM sin diálogos. El estado y los resultados proceden del motor en todos los casos. El modo por archivo admite entradas enteras explícitas y puede guardar un reporte aunque la ejecución termine con error.

La memoria es unificada: código y datos comparten el espacio de direcciones. El cargador impide que las declaraciones iniciales `DATA` se superpongan con el código. Durante la ejecución, `STORE` puede escribir en una dirección válida; si se sobrescribe una instrucción con un dato y luego el PC llega a esa celda, Fetch informa el error.

El estado de error es distinto de la terminación por `HALT`. Un error de ejecución conserva la dirección de la instrucción fallida y bloquea nuevas instrucciones hasta reiniciar o cargar otro programa válido. Cancelar el diálogo de `INPUT` deja pendiente esa instrucción y permite reintentar.

El método interno `reset()` vacía el motor. `restart()` restaura el código cargado y sus datos iniciales, y es la operación de reinicio que utilizan ambas interfaces. Las ediciones de datos antes del primer paso se incorporan a la condición inicial; las posteriores solo afectan a esa ejecución. Los reportes distinguen esas ediciones de las escrituras realizadas por STORE.

Diseño del conjunto de instrucciones

`dir` representa una dirección entera de memoria entre 0 y 99. Las siete operaciones con dirección reciben exactamente un operando. Las tres operaciones restantes no reciben operandos.

Instrucción	Sintaxis	Funcionamiento
Carga	`LOAD dir`	Lee el entero en Mem[dir] y lo copia a ACC.
Almacenamiento	`STORE dir`	Copia ACC a Mem[dir]; conserva ACC.
Suma	`ADD dir`	Calcula ACC = ACC + Mem[dir].
Resta	`SUB dir`	Calcula ACC = ACC - Mem[dir].
Salto incondicional	`JMP dir`	Reemplaza PC por dir.
Salto si cero	`JZ dir`	Si Z es 1, reemplaza PC por dir; si no, continúa.
Salto si no cero	`JNZ dir`	Si Z es 0, reemplaza PC por dir; si no, continúa.
Entrada	`INPUT`	Obtiene un entero del teclado y lo coloca en ACC.
Salida	`OUTPUT`	Emite ACC en la impresora virtual.
Terminación	`HALT`	Activa el estado de detención y finaliza.

El conjunto contiene diez instrucciones. `DATA dir valor` es una directiva de carga, no una instrucción de CPU, y no consume ciclos ni direcciones de código.

Las instrucciones completadas actualizan Z y N según ACC. Los saltos consultan Z antes de esa actualización. Como un salto no modifica ACC, las banderas conservan el mismo valor al terminarlo. No se implementan multiplicación, división, etiquetas simbólicas ni operandos inmediatos como instrucciones del motor. ACC se usa implícitamente; por ejemplo, `LOAD ACC` es inválido porque LOAD recibe una dirección entera, no un registro.

Diseño del ciclo de instrucción

Cada llamada correcta a `step()` completa esta secuencia:

1. Verificar que no haya un error previo ni una terminación por HALT.
2. **Fetch:** leer Mem[PC], comprobar que contiene una instrucción y copiarla a IR.
3. **Decode:** identificar el opcode y su operando.
4. Preparar la continuación secuencial: PC = PC + 1, después de Decode y antes de Execute.
5. **Execute:** aplicar la operación. Un salto puede reemplazar el PC preparado.
6. Actualizar Z y N, incrementar el contador de ciclos y registrar la explicación, las fases y los cambios en el historial de instrucciones completadas.

Si una instrucción falla, el ciclo no se cuenta como completado y PC vuelve a la dirección fallida. Si se cancela INPUT, se restauran PC e IR anteriores; ACC y el contador no cambian. No se agregan entradas, salidas ni pasos de historial por una instrucción incompleta. HALT sí es una instrucción completada: deja IR en HALT, incrementa PC y cuenta un ciclo. Alcanzarlo en el último ciclo permitido es una terminación correcta.

Ejemplo completo: sumar 10 y 20

```
DATA 10 10
DATA 11 20
LOAD 10
ADD 11
STORE 20
OUTPUT
HALT
```

La carga coloca las cinco instrucciones en las direcciones 0 a 4. El estado inicial es PC = 0, IR = NINGUNO, ACC = 0, Z = 1, N = 0 y Mem[20] = 0. Las celdas 10 y 11 contienen 10 y 20.

La siguiente tabla muestra los valores antes y después de cada instrucción. En la columna IR también se explicita el valor anterior.

Instrucción	PC	IR	ACC	Mem[20]
LOAD 10	0 → 1	NINGUNO → LOAD 10	0 → 10	0 → 0
ADD 11	1 → 2	LOAD 10 → ADD 11	10 → 30	0 → 0
STORE 20	2 → 3	ADD 11 → STORE 20	30 → 30	0 → 30
OUTPUT	3 → 4	STORE 20 → OUTPUT	30 → 30	30 → 30
HALT	4 → 5	OUTPUT → HALT	30 → 30	30 → 30

Después de LOAD, las banderas son Z = 0 y N = 0 y conservan esos valores hasta terminar. OUTPUT agrega 30 a la impresora. HALT deja la máquina detenida con cinco ciclos completados. Las celdas 10 y 11 conservan sus datos.

Estado al terminar cada fase

En esta tabla, Fetch y Decode todavía mantienen el PC de la instrucción actual. El incremento secuencial ocurre entre Decode y Execute. La columna de memoria presenta Mem[20]; el código y los otros datos no cambian.

Instrucción	Fase terminada	PC	IR	ACC	Mem[20]
LOAD 10	Fetch	0	LOAD 10	0	0
LOAD 10	Decode	0	LOAD 10	0	0
LOAD 10	Execute	1	LOAD 10	10	0
ADD 11	Fetch	1	ADD 11	10	0
ADD 11	Decode	1	ADD 11	10	0
ADD 11	Execute	2	ADD 11	30	0
STORE 20	Fetch	2	STORE 20	30	0
STORE 20	Decode	2	STORE 20	30	0
STORE 20	Execute	3	STORE 20	30	30
OUTPUT	Fetch	3	OUTPUT	30	30
OUTPUT	Decode	3	OUTPUT	30	30
OUTPUT	Execute	4	OUTPUT	30	30
HALT	Fetch	4	HALT	30	30
HALT	Decode	4	HALT	30	30
HALT	Execute	5	HALT	30	30

La tabla es una explicación de los estados internos, no una función para detener la interfaz entre fases. La traza exportada registra las fases de cada paso y las tarjetas muestran las de la última instrucción.

Diseño de la memoria

La memoria contiene 100 celdas. Sus direcciones válidas son 0 a 99. Antes de cargar un programa se inicializa con ceros. El código se coloca de forma contigua desde la dirección 0, contando únicamente instrucciones ejecutables. Las líneas vacías, los comentarios y DATA no ocupan direcciones de código.

Zona del ejemplo de suma	Contenido inicial	Contenido final
0 a 4	LOAD, ADD, STORE, OUTPUT y HALT	Sin cambios
10	10	10
11	20	20
20	0	30
Resto de celdas	0	0

DATA exige dos enteros: dirección y valor. Se rechazan direcciones inválidas, direcciones repetidas y declaraciones que coincidan con una celda del código inicial. La validación del programa se completa antes de

reiniciar la memoria; una carga rechazada conserva el programa anterior.

Los accesos fuera del rango generan `MemoryOutOfBoundsException`. Las lecturas aritméticas de una instrucción como si fuera un número generan `InvalidInstructionException`. Un PC que alcance una celda de datos produce un error de Fetch. La capacidad de 100 celdas y el uso de enteros de Python son decisiones didácticas; no se simulan ancho de palabra, desbordamiento aritmético ni jerarquía de caché.

Lenguaje de instrucciones

Un archivo `.asm` es texto UTF-8 con una instrucción o directiva por línea. Los opcodes se interpretan sin distinguir mayúsculas y minúsculas. Se permiten comentarios que comiencen por `#` o `//`, tanto en una línea propia como después de una operación.

```
# Datos iniciales: no consumen direcciones de código
DATA 90 10
DATA 91 20
LOAD 90       // dirección de código 0
ADD 91        // dirección de código 1
STORE 92      // dirección de código 2
OUTPUT        // dirección de código 3
HALT          // dirección de código 4
```

Los operandos de las instrucciones son direcciones, no valores inmediatos. ADD 91 suma el contenido de la celda 91; no suma el número 91. Los destinos de salto también son direcciones de instrucciones y no números de línea del archivo. Por ejemplo, comentarios y declaraciones DATA no cambian el destino de JZ 6.

El cargador rechaza programas vacíos, instrucciones desconocidas, cantidad incorrecta de operandos, operandos no enteros, direcciones fuera de rango, más de 100 instrucciones y programas sin HALT. Que un archivo contenga HALT no demuestra que vaya a alcanzarlo; el límite de ciclos controla los bucles durante la ejecución.

El mismo lenguaje se ejecuta sin interacción desde terminal:

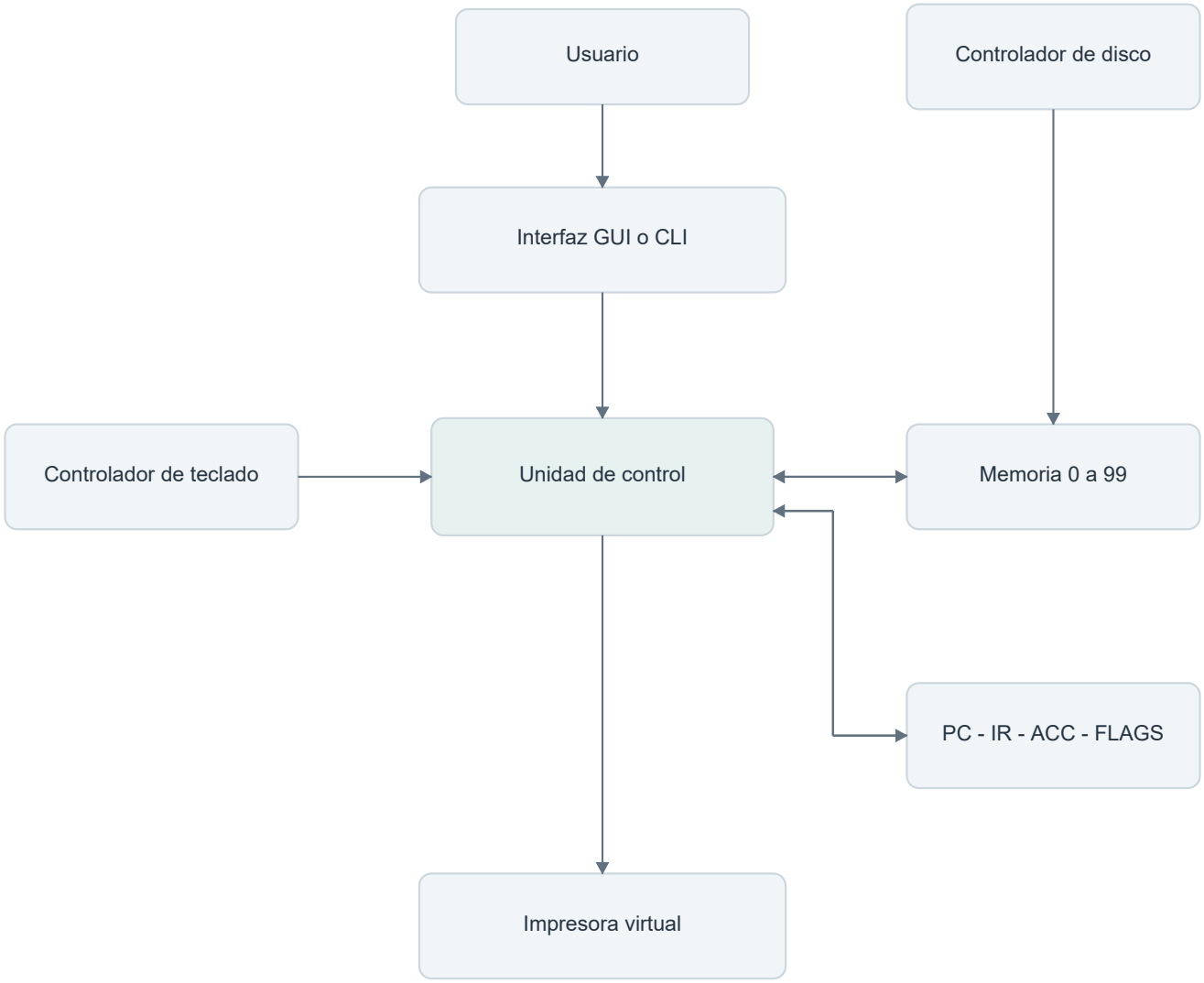
```
python simulador_vm.py --archivo examples/test6_entrada_salida.asm --entrada 7 --reporte
    output/reportes/entrada.html
```

Se repite `--entrada` por cada INPUT. Si faltan valores, la ejecución se detiene con un diagnóstico y no solicita teclado. `--max-ciclos` limita el total desde carga o reinicio, incluyendo pasos previos. El comando devuelve 0 al completar correctamente y 1 ante una falla de carga, ejecución o guardado del reporte.

Diagramas de funcionamiento

Diagrama de arquitectura

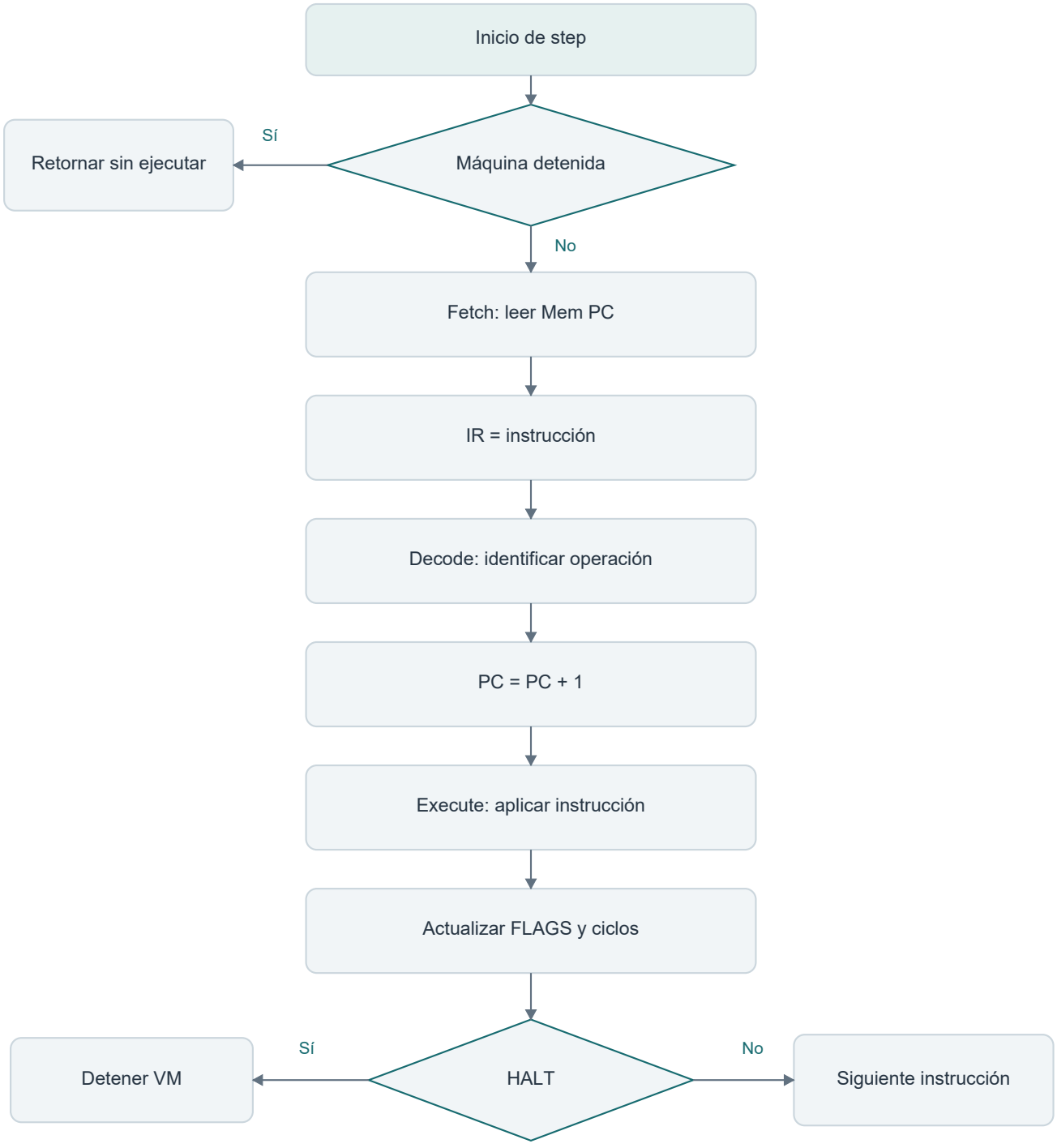
Figura 1. Arquitectura del simulador.



La interfaz utiliza la unidad de control; esta consulta la memoria y actualiza registros. El disco carga programas, el teclado entrega enteros y la impresora recibe los valores emitidos.

Diagrama de flujo de ejecución

Figura 2. Flujo de ejecución de una instrucción.



Si una fase falla: conservar el PC de la instrucción y pasar a ERROR.

El flujo representa una instrucción completada. Un error previo impide entrar al ciclo hasta reiniciar. Un fallo durante el ciclo conserva el PC de la instrucción fallida y activa el diagnóstico de ERROR. La cancelación de entrada conserva la instrucción pendiente para reintentar.

Diagrama del ciclo Fetch-Decode-Execute

Figura 3. Secuencia Fetch-Decode-Execute.

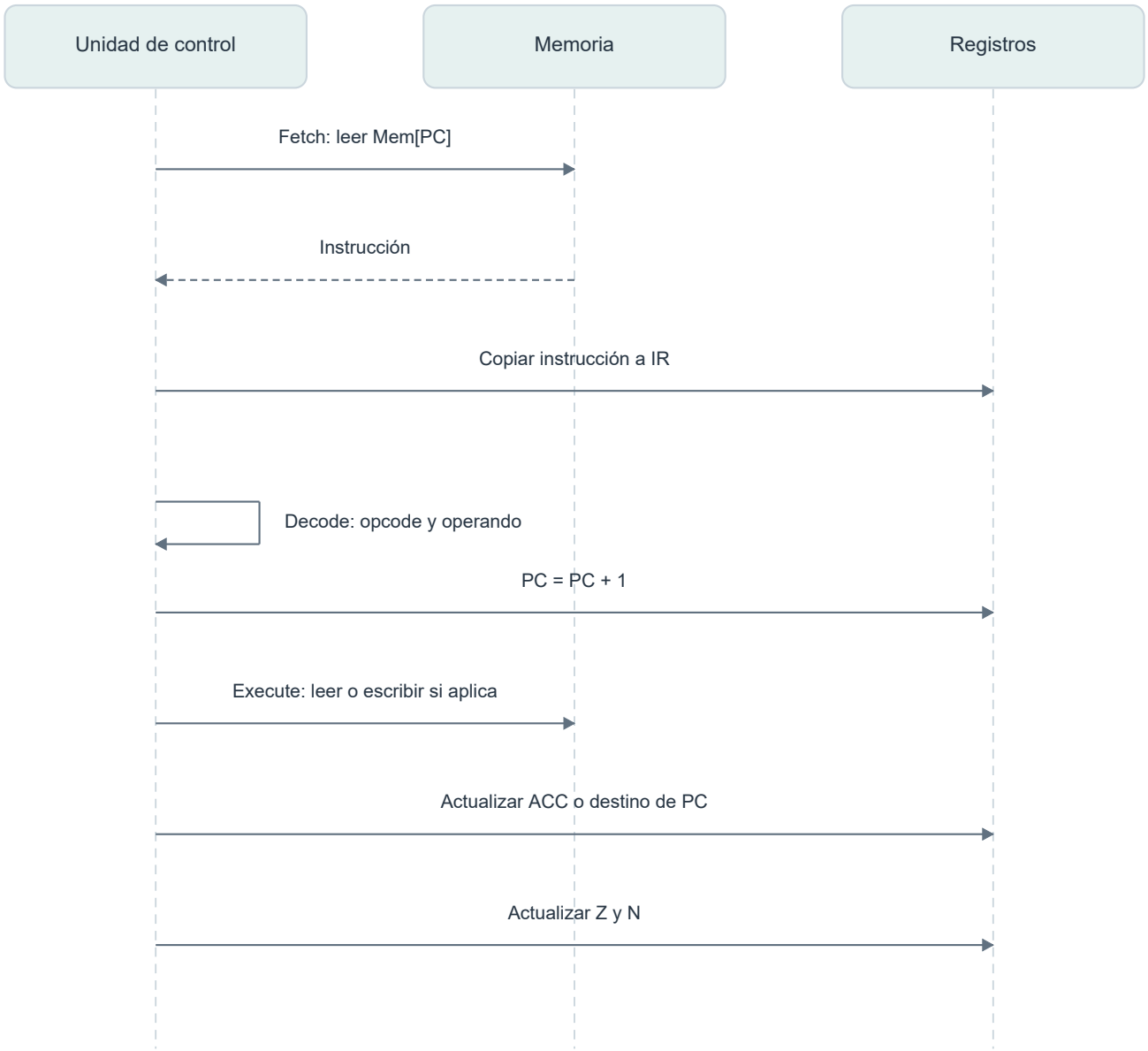
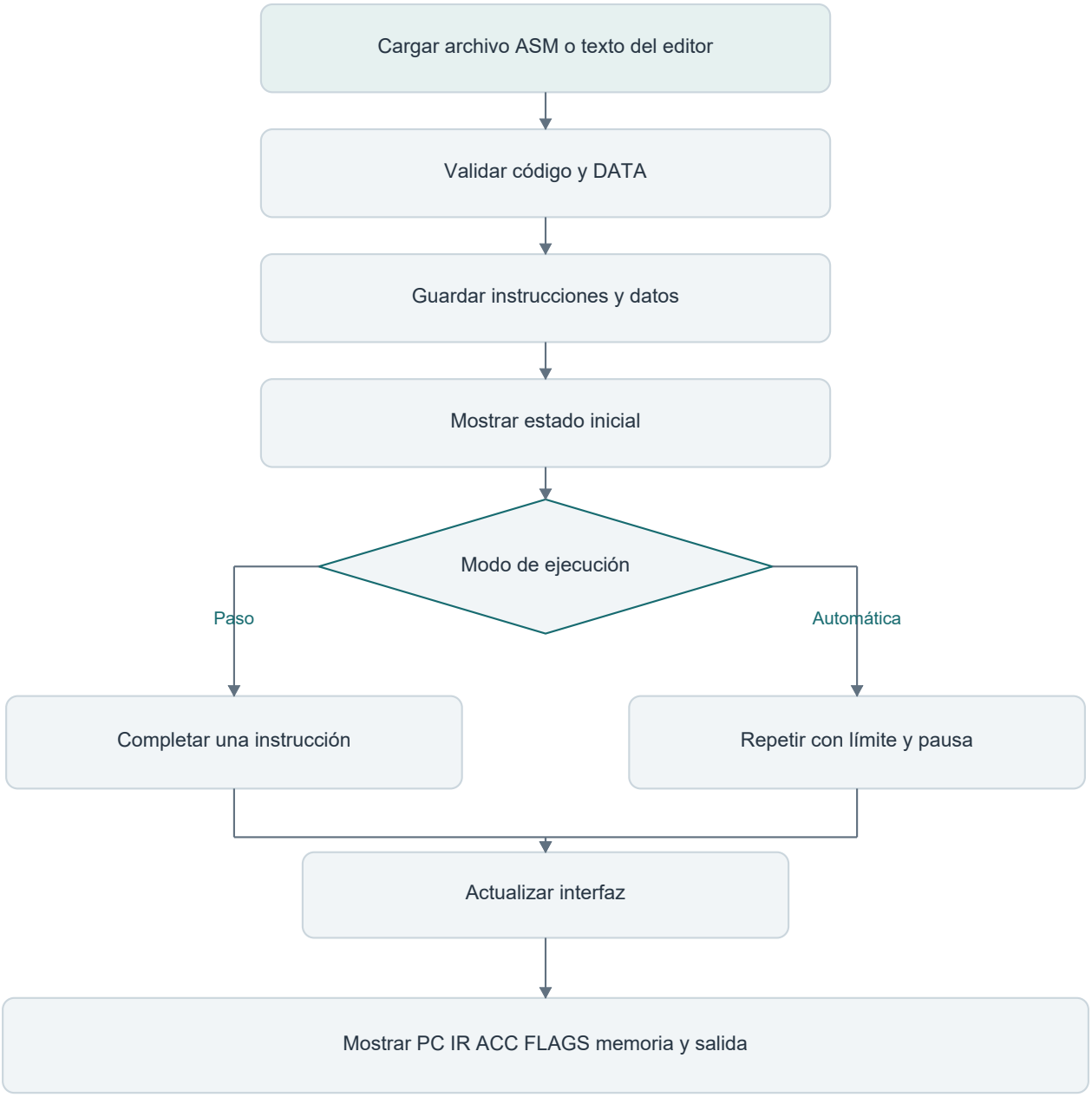


Diagrama general de funcionamiento

Figura 4. Funcionamiento general.



En la versión impresa los cuatro bloques se presentan como diagramas dibujados. El Markdown conserva sus fuentes para facilitar modificaciones futuras.

Casos de prueba

Los siguientes seis programas se incluyen completos para que el informe sea autocontenido. Los archivos equivalentes están en `examples/`. Cada prueba comienza con una carga nueva; las celdas no declaradas empiezan en cero. Las declaraciones iniciales y los destinos de salto se interpretan según las reglas del lenguaje descritas anteriormente.

Caso	Archivo	Resultado esperado
Suma	`examples/test1_suma.asm`	ACC = 30; Mem[20] = 30; salida 30.

Caso	Archivo	Resultado esperado
Resta	`examples/test2_resta.asm`	ACC = 20; Mem[20] = 20; salida 20.
Almacenamiento	`examples/test3_almacenamiento.asm`	Mem[20] = 30; ACC = 30; salida 30.
Condicional	`examples/test4_condicional.asm`	JZ toma el destino 6; ACC = 111; salida 111.
Bucle	`examples/test5_bucle.asm`	Calcula 3 + 2 + 1; Mem[53] = 6; salida 6.
Entrada y salida	`examples/test6_entrada_salida.asm`	Con entrada 7: ACC = 7; Mem[20] = 7; salida 7.

Caso 1: suma

Datos de entrada: Mem[10] = 10 y Mem[11] = 20, declarados con DATA. No pide teclado.

```
DATA 10 10
DATA 11 20

# Prueba 1 - Suma
# Datos iniciales: Mem[10] = 10, Mem[11] = 20
# Esperado: ACC = 30 y Mem[20] = 30
LOAD 10
ADD 11
STORE 20
OUTPUT
HALT
```

Resultado esperado: ACC = 30, Mem[20] = 30, impresora = [30], PC = 5, IR = HALT y máquina detenida. La tabla del ciclo anterior desarrolla esta prueba instrucción por instrucción.

Caso 2: resta

Datos de entrada: Mem[10] = 50 y Mem[11] = 30. No pide teclado.

```
DATA 10 50
DATA 11 30

# Prueba 2 - Resta
# Datos iniciales: Mem[10] = 50, Mem[11] = 30
# Esperado: ACC = 20 y Mem[20] = 20
LOAD 10
SUB 11
STORE 20
OUTPUT
HALT
```

Resultado esperado: ACC = 20, Mem[20] = 20, impresora = [20], PC = 5, IR = HALT y máquina detenida. Comprueba el orden de la resta: 50 - 30.

Caso 3: almacenamiento

Datos de entrada: Mem[10] = 30. Mem[20] inicia en cero. No pide teclado.

```
DATA 10 30

# Prueba 3 - Almacenamiento
# Datos iniciales: Mem[10] = 30
# Esperado: Mem[20] = 30
LOAD 10
STORE 20
LOAD 20
OUTPUT
HALT
```

Resultado esperado: Mem[20] = 30, ACC = 30, impresora = [30], PC = 5, IR = HALT y máquina detenida. El segundo LOAD comprueba que el dato almacenado puede recuperarse.

Caso 4: salto condicional

Datos de entrada: Mem[10] = 20, Mem[11] = 20, Mem[12] = 111 y Mem[13] = 999. No pide teclado.

```
DATA 10 20
DATA 11 20
DATA 12 111
DATA 13 999

# Prueba 4 - Condicional con JZ
# Datos iniciales: Mem[10] = 20, Mem[11] = 20, Mem[12] = 111, Mem[13] = 999
# Esperado: ACC = 111 porque 20 - 20 activa la bandera Z
LOAD 10
SUB 11
JZ 6
LOAD 13
OUTPUT
HALT
LOAD 12
OUTPUT
HALT
```

Resultado esperado: la resta produce cero y activa Z. JZ 6 salta a LOAD 12; no se ejecuta la rama que imprimiría 999. Termina con ACC = 111, impresora = [111], PC = 9 e IR = HALT.

Caso 5: bucle con JNZ

Datos de entrada: Mem[50] = 3, Mem[51] = 1 y Mem[52] = 0. No pide teclado. Usa Mem[54] como contador y Mem[53] como suma parcial.

```
DATA 50 3
DATA 51 1
DATA 52 0

# Prueba 5 - Bucle con JNZ
# Datos iniciales: Mem[50] = 3, Mem[51] = 1, Mem[52] = 0
# Calcula 3 + 2 + 1
# Esperado: ACC = 6 y Mem[53] = 6
LOAD 50
STORE 54
LOAD 52
STORE 53
LOAD 53
ADD 54
STORE 53
LOAD 54
SUB 51
STORE 54
JNZ 4
LOAD 53
OUTPUT
HALT
```

Resultado esperado: el contador toma los valores 3, 2, 1 y 0. Cuando llega a cero, JNZ deja de saltar. Termina con ACC = 6, Mem[53] = 6, Mem[54] = 0, impresora = [6], PC = 14 e IR = HALT.

Caso 6: entrada y salida

Dato de entrada por teclado: 7. No requiere declaraciones DATA.

```
# Prueba 6 - Entrada y salida
# Entrada sugerida: 7
# Esperado: ACC = 7 y Mem[20] = 7
INPUT
STORE 20
OUTPUT
HALT
```

Resultado esperado: INPUT coloca 7 en ACC, STORE lo conserva en Mem[20] y OUTPUT lo muestra. Termina con ACC = 7, Mem[20] = 7, impresora = [7], PC = 4 e IR = HALT.

Pruebas de errores y evidencia obtenida

Las pruebas automatizadas complementan los programas anteriores con instrucciones inexistentes, ausencia de HALT, direcciones inválidas, sintaxis incorrecta, carga inválida sin pérdida del programa anterior, banderas, saltos, cancelación de entrada, reinicio y límites de ciclos. El archivo `tests/verificar_entrega.py` y las pruebas en `tests/` contienen las comprobaciones ejecutables.

Los resultados realmente obtenidos se documentan en `docs/EVIDENCIAS_PRUEBAS.md`. Los JSON de `docs/evidencias/` conservan programa, datos, registros, memoria, impresora y trazas. Para actualizar esa evidencia se ejecuta desde la raíz:

```
python scripts/generar_evidencias.py --incluir-gui
```

La opción `--incluir-gui` también verifica controles de Tkinter y necesita una sesión de escritorio. Sin esa opción se generan las evidencias y comprobaciones que no requieren abrir ventanas. El informe de evidencias identifica qué comprobaciones se ejecutaron y su resultado; no se sustituye por una afirmación fija en este documento.

Resultados esperados

El producto debe cargar programas válidos, conservar instrucciones y datos dentro del rango de memoria y ejecutar las diez operaciones con los cambios descritos. Cada instrucción completada debe actualizar el estado observable de forma coherente: PC e IR deben permitir seguir el flujo, ACC y las banderas deben reflejar el cálculo y STORE debe modificar la celda indicada.

La suma, resta y almacenamiento deben producir los valores de los casos 1 a 3. Los saltos deben seleccionar la rama o repetir el cuerpo según la bandera Z, como muestran los casos 4 y 5. El teclado y la impresora deben permitir que el caso 6 lea y reporte el mismo entero.

Una instrucción de terminación debe detener la ejecución. Un error debe identificar su causa y conservar un diagnóstico visible; no debe permitir seguir ejecutando una máquina en estado de error. El límite de ciclos debe detener un bucle que no alcance HALT. La presencia de HALT en el texto solo satisface la regla del cargador, no garantiza terminación.

Los resultados obtenidos y su validación se adjuntan en el informe de evidencias. La aplicación utiliza la biblioteca estándar de Python; ReportLab solo se necesita para regenerar la versión PDF de la documentación y no forma parte del motor de la máquina virtual.

Reportes y trazabilidad

El reporte se construye como una copia independiente del estado actual. Contiene nombre y código del programa, fecha, datos iniciales, registros, memoria completa, entradas, valores impresos, historial por instrucción, ediciones manuales y estadísticas de operaciones. Cada paso identifica PC y ACC antes y después, banderas, cambios de memoria y fases del ciclo. El campo de últimas fases describe únicamente la última instrucción intentada.

HTML ofrece una presentación autónoma para leer e imprimir; CSV utiliza filas con tipo de registro y separador de punto y coma para su revisión en Excel; JSON conserva los datos estructurados. La exportación identifica si el estado es LISTA, PAUSADA, FINALIZADA o ERROR. Un estado exportado pausado o con error no se presenta como una prueba aprobada. El guardado reemplaza el destino al completar la escritura para evitar truncarlo ante un error previo.

Conclusiones

La implementación representa una arquitectura sencilla en la que una memoria unificada, tres registros principales, dos banderas y una unidad de control permiten ejecutar programas completos. El conjunto de diez instrucciones cubre transferencia, aritmética, control del flujo y entrada/salida.

La traza numérica de la suma permite relacionar cada fase con cambios concretos en PC, IR, ACC y memoria. Los seis programas propuestos permiten contrastar resultados calculados a mano con el comportamiento del simulador. Las comprobaciones automatizadas y los archivos exportados aportan evidencia que puede repetirse desde el código entregado.

El modelo simplifica una computadora real: no simula interrupciones, concurrencia, protección por procesos, ancho de palabra ni todos los dispositivos físicos. Estas limitaciones delimitan el alcance didáctico del proyecto y no impiden demostrar el ciclo de instrucciones solicitado.

Recomendaciones

- Completar con información real la institución, integrantes, docente y fecha de entrega de la portada.
- Mantener constancia de la autorización de Python ante la diferencia de redacción de la propuesta.
- Regenerar evidencias después de modificar el motor o los programas, y revisar el resultado de cada comprobación.
- Revisar la ventana en la resolución de la computadora de defensa y comprobar allí Tkinter, archivos y atajos.
- Imprimir el PDF generado con los diagramas dibujados y revisar su vista previa antes de entregar.
- Ensayar una exposición de 15 a 20 minutos que incluya suma por pasos, salto, entrada/salida y un error controlado.
- Distribuir la exposición entre tres integrantes y practicar las transiciones utilizando el guion y las diapositivas de la entrega.
- Conservar un reporte de suma y otro de error para explicar cómo se documenta un resultado sin depender de una ejecución en vivo.

Bibliografía

- Propuesta de proyecto del grupo 13. *Implementación de una máquina virtual simple*. Archivo proporcionado: `Propuesta #13..pdf`. Se emplea como especificación de entregables.
- Tanenbaum, A. S. y Bos, H. *Modern Operating Systems*.
- Silberschatz, A., Galvin, P. B. y Gagne, G. *Operating System Concepts*.
- Stallings, W. *Computer Organization and Architecture*.
- Python Software Foundation. *Documentación de Python 3*. https://docs.python.org/3/
- Python Software Foundation. *tkinter: Python interface to Tcl/Tk*. https://docs.python.org/3/library/tkinter.html

Las referencias bibliográficas se presentan sin años ni datos editoriales no confirmados.

Anexo A. Manual de usuario

Fecha de revisión: 27 de octubre de 2026.

Requisitos e inicio

Utiliza Python 3.10 o superior con Tkinter. La aplicación no requiere paquetes externos. El proyecto conserva Python según autorización consignada por el grupo.

Abre una terminal en la raíz del proyecto y ejecuta:

```
python simulador_vm.py
```

Para utilizar el menú de consola:

```
python simulador_vm.py --cli
```

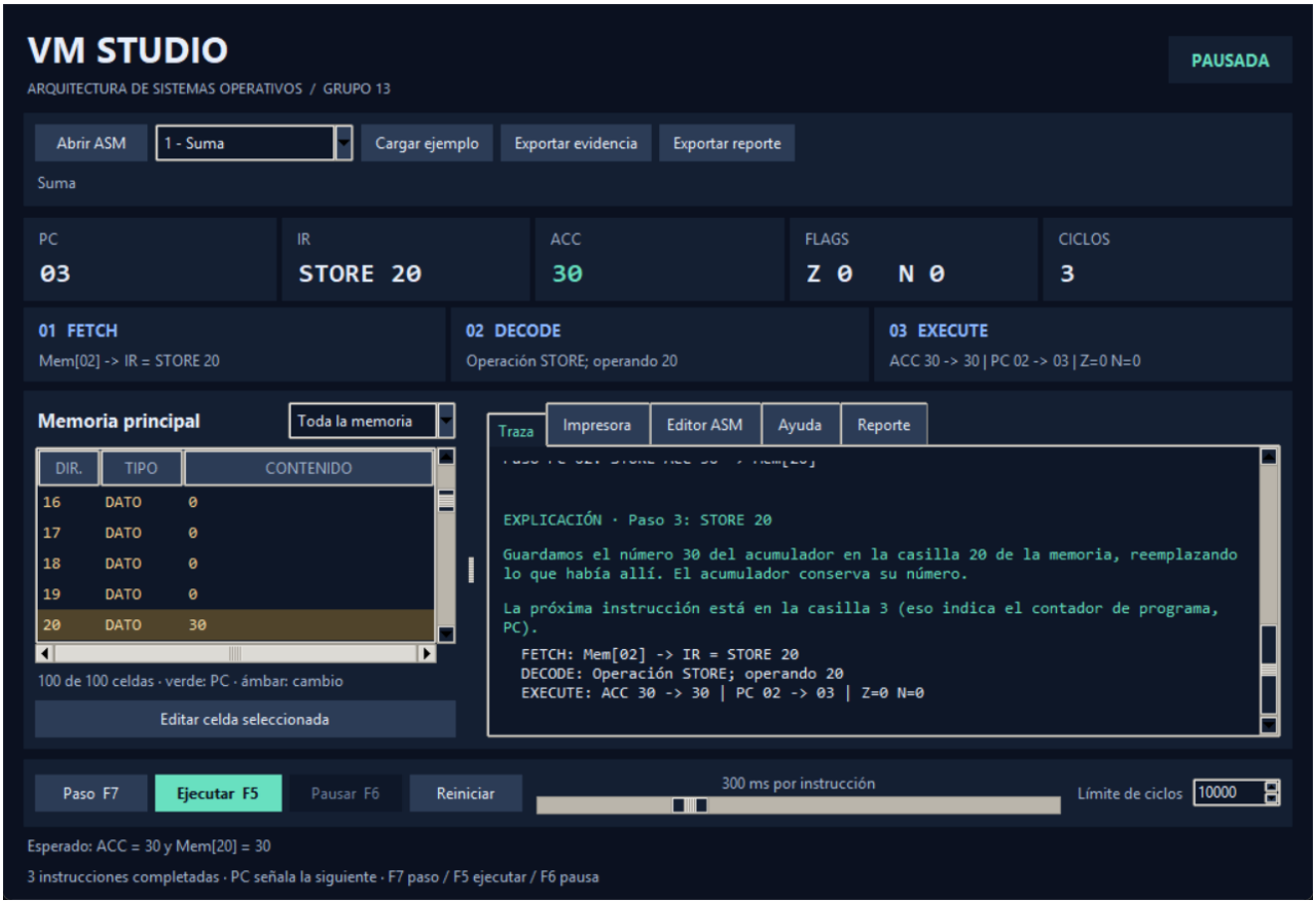
En Windows puedes sustituir `python` por `py` si ese es el comando disponible. `python -m tkinter` permite comprobar si la instalación dispone de Tkinter. ReportLab solo se utiliza para regenerar la documentación PDF y no hace falta para abrir el simulador.

También puedes abrir `INICIAR_VM.cmd` con doble clic. Si recibiste un ZIP, primero descomprime la carpeta completa. No ejecutes el archivo dentro del ZIP ni separes `simulador_vm.py` de `src/` y `examples/`.

Para consultar todas las opciones disponibles, ejecuta `python simulador_vm.py --help`. Para cerrar la ventana utiliza su botón de cierre; en consola selecciona 0.

Elementos de la interfaz gráfica

- **Estado:** indica si la máquina espera un programa, está lista, ejecutándose, pausada, finalizada o con error.
- **Registros:** muestran PC, IR, ACC, FLAGS y ciclos completados. PC señala la siguiente instrucción; la fila destacada en memoria corresponde a esa dirección.
- **Fases:** muestran Fetch, Decode y Execute de la última instrucción. Un paso ejecuta las tres fases completas.
- **Memoria:** contiene 100 celdas, de 0 a 99. Sus filtros permiten revisar toda la memoria, instrucciones, datos o datos distintos de cero. Los cambios de memoria recientes se distinguen del siguiente PC.
- **Traza:** registra operaciones, cambios de estado y explicaciones en palabras simples.
- **Impresora:** reúne los valores emitidos por OUTPUT.
- **Editor ASM:** permite escribir, guardar y cargar un programa. Modificar el texto no cambia el programa activo hasta cargarlo; revisar el aviso de cambios pendientes antes de ejecutar.
- **Reporte:** resume el estado y el historial; se puede exportar HTML, CSV o JSON.
- **Ayuda:** resume instrucciones y atajos.



Interfaz de VM Studio después de STORE 20: acumulador 30 y memoria 20 modificada.

Primera ejecución: suma de 10 y 20

1. Selecciona el ejemplo Suma y pulsa Cargar ejemplo, o pega el siguiente código en Editor ASM y pulsa Cargar desde editor.
2. Comprueba que PC y ACC están en cero y las celdas 10 y 11 contienen 10 y 20.
3. Pulsa Paso o F7. LOAD 10 deja ACC en 10.
4. Pulsa Paso otra vez. ADD 11 deja ACC en 30.
5. Completa los pasos restantes. STORE 20 guarda 30, OUTPUT imprime 30 y HALT termina.

```
DATA 10 10
DATA 11 20
LOAD 10
ADD 11
STORE 20
OUTPUT
HALT
```

El resultado final debe ser PC = 5, IR = HALT, ACC = 30, Mem[20] = 30 e impresora con 30. Las tarjetas de fases corresponden entonces a HALT; la pestaña Traza conserva también la explicación de las instrucciones anteriores.

Reglas para escribir programas

Cada línea contiene una operación. LOAD, STORE, ADD, SUB, JMP, JZ y JNZ requieren una dirección entre 0 y 99. INPUT, OUTPUT y HALT no reciben operandos. Todo programa debe incluir HALT.

`DATA dirección valor` coloca un entero inicial en memoria. No cuenta como instrucción, dirección de código ni ciclo. No se permiten declaraciones repetidas o sobre el código inicial. Las instrucciones empiezan en la dirección cero y se cuentan sin incluir comentarios o DATA; los saltos usan esas direcciones, no los números de línea del archivo.

En `ADD 11`, 11 es una dirección: se suma el número almacenado allí. Para sumar un valor fijo se declara previamente con DATA. ACC se utiliza implícitamente; `LOAD ACC` no es una sintaxis válida. Los comentarios pueden comenzar con `#` o `//`.

Controles de ejecución

Control	Acción
Paso / F7	Completa una instrucción y actualiza el estado.
Ejecutar / F5	Inicia o continúa la ejecución automática.
Pausar / F6	Detiene la programación de las siguientes instrucciones.
Reiniciar	Restaura el programa gráfico y sus datos iniciales.
Velocidad	Ajusta la espera en milisegundos entre instrucciones automáticas.
Límite de ciclos	Limita los ciclos totales completados desde la carga o reinicio.

Los pasos manuales también cuentan para el límite de la GUI. Si HALT se completa en el último ciclo permitido, la terminación es correcta. Un programa puede contener HALT y repetir un salto sin llegar a él; por eso existe el límite.

Después de HALT, utiliza Reiniciar para volver a ejecutar. Una edición de datos antes del primer paso se conserva como dato inicial al reiniciar; una edición después de empezar solo afecta al recorrido actual. Para cambiar una instrucción usa el editor y vuelve a cargar el código.

Teclado y salida

El ejemplo Entrada y salida pide un entero. Introduce 7 y completa el programa para comprobar ACC = 7, Mem[20] = 7 e impresora con 7.

Si cancelas el diálogo de INPUT, PC, ACC y el contador de ciclos se mantienen y la instrucción queda pendiente. Pulsa Paso o Ejecutar para reintentar. Cancelar la entrada no exige reiniciar.

Errores y recuperación

Situación	Diagnóstico y solución
Archivo no encontrado	Comprueba la ruta del ASM; el programa anterior permanece cargado.
Instrucción desconocida o operandos incorrectos	Corrige la línea indicada y vuelve a cargar.
LOAD ACC u otro registro como operando	ACC se utiliza implícitamente. Escribe una dirección entera; por ejemplo, LOAD 10.
DATA repetido, sobre código o fuera de rango	Elige una dirección de datos libre entre 0 y 99.
Programa sin HALT	Añade la terminación y vuelve a cargar.
Lectura de código como dato	Revisa la dirección usada por LOAD, ADD o SUB.
Fetch encuentra un dato	Revisa el destino de salto o si STORE sobrescribió código.
Límite de ciclos alcanzado	Revisa el bucle o aumenta el límite si la duración es intencionada; después reinicia.
Estado ERROR	Lee el diagnóstico, corrige la causa y reinicia o carga un programa válido.

Un error de ejecución conserva PC en la instrucción fallida y bloquea pasos posteriores. La consola también conserva y muestra el diagnóstico al consultar registros. Una carga inválida no borra el programa previo.

Archivos y evidencias

Guardar ASM guarda el texto actual del editor. Para ejecutarlo primero debes cargarlo. Abrir un archivo o cargar un ejemplo instala su código y sus datos iniciales.

Exportar evidencia guarda un JSON con código, datos iniciales, registros, memoria, entradas, impresora, historial, ediciones manuales, traza y fases. Exportar reporte permite elegir HTML para abrir o imprimir en un navegador, CSV para Excel o JSON. La pestaña de reporte resume el estado y el historial de la ejecución. Exporta después de completar un caso para documentar su resultado; también puedes exportar una pausa o un error, que quedarán identificados con ese estado.

Un reporte es una instantánea: seguir ejecutando no modifica el archivo ya guardado. HTML funciona sin conexión. En CSV, las filas incluyen un campo de tipo para distinguir registros, pasos, memoria, entradas, salidas y ediciones. Al exportar no se abre automáticamente un navegador.

Para regenerar las evidencias de entrega desde terminal:

```
python scripts/generar_evidencias.py --incluir-gui
```

El informe generado queda en `docs/EVIDENCIAS_PRUEBAS.md` y los archivos de detalle en `docs/evidencias/`. La opción de GUI necesita una sesión de escritorio. Sin `--incluir-gui` se evita abrir ventanas y se identifica esa condición en el resultado.

Uso de la consola

El menú permite cargar uno de los seis casos o un archivo, ejecutar un paso, ejecutar todo, consultar registros y memoria, modificar una celda y reiniciar. Los archivos de ejemplo están en `examples/`.

La opción Ejecutar todo utiliza por defecto un límite total de 10000 ciclos desde carga o reinicio. También cuentan los pasos manuales; pausar o reanudar no restablece el contador. Si aparece ERROR, la consulta de registros muestra el diagnóstico y debes reiniciar o cargar un programa válido antes de seguir.

La opción Reiniciar de consola restaura el programa cargado y sus datos iniciales, igual que el botón gráfico. Una edición de datos anterior al primer paso se incorpora a esos datos iniciales; una edición posterior solo afecta a esa ejecución. La opción 9 exporta el reporte indicando una ruta con extensión `.html`, `.csv` o `.json`. EOF y Ctrl+C cierran la sesión sin mostrar un error interno de Python.

Ejecución por archivo sin interacción

Desde la raíz del proyecto:

```
python simulador_vm.py --archivo examples/test1_suma.asm --reporte output/reportes/suma.html
python simulador_vm.py --archivo examples/test6_entrada_salida.asm --entrada 7 --reporte
    output/reportes/entrada.csv
python simulador_vm.py --archivo examples/multiplicacion.asm --entrada -4 --entrada 3 --reporte
    output/reportes/multiplicacion.json
```

Cada `--entrada ENTERO` corresponde a un INPUT en orden. Si falta un valor, el programa informa el error y termina; no espera teclado ni abre ventanas. Con `--reporte` también puedes conservar ese diagnóstico. `--max-ciclos 10000` fija el límite total. Un recorrido correcto devuelve código de salida 0; un error de carga, ejecución o guardado devuelve 1. `python simulador_vm.py --help` muestra las opciones.

Material para exponer y compartir

El PowerPoint se encuentra en `output/presentacion/DEFENSA_GRUPO13.pptx`. El guion editable para tres integrantes está en `output/exposicion/GUIA_TRES_INTEGRANTES.docx` y su fuente en `docs/exposicion/GUIA_TRES_INTEGRANTES.md`. La defensa debe durar de 15 a 20 minutos e incluir una ejecución por pasos, automática y un error controlado.

El documento técnico editable está en `output/word/ENTREGA_GRUPO13.docx` y su versión imprimible en `output/pdf/ENTREGA_GRUPO13.pdf`. Los datos de portada deben completarse con la información real del grupo. Para compartir código y materiales juntos, utiliza `output/compartir/VM_STUDIO_GRUPO13.zip`; debe descomprimirse antes de ejecutar.

Ejemplo adicional de multiplicación

`examples/multiplicacion.asm` multiplica mediante sumas sucesivas. Solicita dos enteros: el primero puede tener cualquier signo; el segundo debe ser no negativo. Introducir cero produce cero. Un segundo entero negativo no satisface la condición del algoritmo y provoca un bucle que detiene el límite de ciclos.

Anexo B. Manual técnico

Fecha de revisión: 26 de octubre de 2026.

Entorno y alcance

La aplicación requiere Python 3.10 o superior. Su motor, consola y GUI utilizan la biblioteca estándar; la interfaz gráfica necesita que Python incluya Tkinter. El proyecto conserva Python según autorización consignada por el grupo. La propuesta también contiene referencias a Java; la autorización se registra como antecedente del grupo, sin afirmar una verificación externa.

ReportLab es una dependencia opcional para generar el documento PDF. La generación Word utiliza python-docx y pypdfium2 para las imágenes del documento. No se necesitan estos paquetes para ejecutar la máquina virtual, sus reportes ni sus pruebas del motor.

Estructura del proyecto

```
simulador_vm.py                 Punto de entrada
src/
  vm_studio/
    __main__.py                 Entrada como módulo
    simulador_vm.py             API pública compatible
    motor.py                    Memoria, CPU y controladores
    interfaz.py                 Interfaz Tkinter y planificación
    consola.py                  Menú y argumentos de terminal
    reportes.py                 Instantáneas y exportación de reportes
examples/
  test1_suma.asm                Suma con datos iniciales
  test2_resta.asm               Resta
  test3_almacenamiento.asm      STORE y lectura posterior
  test4_condicional.asm         Salto JZ
  test5_bucle.asm               Bucle JNZ
  test6_entrada_salida.asm      INPUT y OUTPUT
  multiplicacion.asm            Ejemplo adicional por sumas sucesivas
tests/
  verificar_entrega.py          Verificación funcional de entrega
  test_regresiones.py           Pruebas de regresión del motor
  test_consola.py               Flujos de terminal y ejecución por archivo
  test_reportes.py              Exportación, integridad y formatos
  verificar_gui.py              Comprobaciones de controles Tkinter
scripts/
  generar_evidencias.py         Resultados y evidencias reproducibles
  generar_documento.py          Documento PDF con diagramas dibujados
  generar_word.py               Documento técnico y guion editables
  preparar_entrega.py           Paquete seleccionado para compartir
  requirements_documento.txt    Dependencia de generación del PDF
docs/
  DOCUMENTO_TECNICO_GRUPO13.md   Diseño y seis programas completos
  MANUAL_USUARIO.md             Uso de la aplicación
  MANUAL_TECNICO.md             Este documento
  EVIDENCIAS_PRUEBAS.md          Resultados obtenidos
  GUIA_DEFENSA.md               Exposición y demostración
  README_ENTREGA.md             Contenido de la entrega
  evidencias/                   Exportaciones JSON y resultados
  exposicion/                   Guion para tres integrantes y recursos
output/
  pdf/
    ENTREGA_GRUPO13.pdf         Versión preparada para imprimir
  word/                         Documento técnico editable
  exposicion/                   Guía editable para tres integrantes
  presentacion/                 Diapositivas editables con imágenes
  reportes/                     Ejemplos de reportes HTML, CSV y JSON
  compartir/                    ZIP de la entrega
```

La implementación oficial se encuentra en `src/vm_studio/`. El archivo `simulador_vm.py` de la raíz funciona como entrada a ese módulo. El material histórico de `archive/` no es la versión de entrega.

Modelo de datos

`Instruction` representa una operación mediante `opcode` en mayúsculas y `operand` opcional. Su representación textual conserva una forma como `LOAD 10` o `HALT`.

`Program` amplía una lista de instrucciones y añade `initial_data`, un diccionario que recoge las declaraciones DATA, y `source`, el texto original. Una declaración no se inserta en la lista de instrucciones y no cambia sus direcciones. `Instruction.validate()` comprueba opcode, cantidad y tipo de operandos antes de cargar o ejecutar una instrucción.

`Memory` conserva 100 celdas inicializadas a cero. Sus operaciones son `read(address)`, `read_int(address)`, `write(address, value)`, `_check_bounds(address)` y `clear()`. `read_int()` rechaza instrucciones; `_check_bounds()` exige

un entero de 0 a 99 y rechaza tipos como booleanos, decimales y texto. La escritura admite enteros o instrucciones válidas. STORE puede modificar código, por lo que ejecutar después una celda sobrescrita con un entero produce un error de Fetch. La edición manual de la interfaz utiliza `edit_data()` y no permite escribir sobre código.

Controladores

`DeviceControllers` ofrece las funciones compartidas de entrada, salida, lectura de archivos y análisis del texto. `KeyboardController.read`, `PrinterController.write` y `DiskController.load` exponen los controladores específicos.

El teclado recibe un entero; en consola solicita otra entrada cuando el texto no representa un entero. En GUI se usa un diálogo. Cancelar el diálogo genera `InputCancelled` y permite repetir la instrucción.

El disco lee archivos con codificación `utf-8-sig`, compatible con UTF-8 con o sin marca inicial. `DiskController.read_source()` obtiene el texto para abrirlo en el editor; `DiskController.load()` lo analiza para cargar el motor. La impresora entrega mensajes a la consola o al manejador de salida de la GUI. No se comunica con una impresora física.

Máquina virtual

`VirtualMachine`, en `motor.py`, conserva `memory`, `pc`, `ir`, `acc`, `flag_z`, `flag_n`, `halted`, `error`, `cycles`, `execution_log` y `last_trace`. También conserva `program_source`, `program_name`, `initial_data`, `input_values`, `outputs`, `history` y `manual_edits`. Puede recibir `input_handler` y `output_handler` para desacoplar el motor de los dispositivos de una interfaz concreta.

Método	Responsabilidad
`reset()`	Vaciar memoria, traza y estado; volver a PC = 0, ACC = 0, Z = 1 y N = 0.
`restart()`	Restaurar el código y los datos iniciales del programa cargado; limpiar su ejecución.
`edit_data(address, value)`	Editar un entero fuera del código y registrar su cambio; antes del primer paso lo incorpora a los datos iniciales.
`load_program(filepath)`	Leer un archivo mediante el controlador de disco y cargarlo.
`load_source(source, name)`	Analizar y cargar un texto ASM.
`load_instructions(instructions, filepath)`	Validar programa y datos iniciales antes de sustituir el estado anterior.
`step()`	Completar un ciclo Fetch-Decode-Execute y registrar su explicación.
`_execute(instruction, original_pc)`	Aplicar el efecto del opcode.
`_update_flags()`	Calcular Z y N a partir de ACC.
`run_all(max_cycles=10000)`	Repetir pasos hasta HALT o un fallo, limitando el total desde carga o reinicio.
`get_state_summary()`	Devolver registros, banderas, detención, error y ciclos.
`print_state()`	Mostrar el estado y el diagnóstico de error en consola.

Carga y validación

El analizador elimina comentarios `#` y `//`, separa tokens y normaliza el opcode. Comprueba que cada instrucción reciba exactamente los operandos previstos y que sus direcciones sean enteros válidos. Una entrada como `LOAD ACC` genera un diagnóstico específico: ACC se utiliza implícitamente y LOAD necesita una dirección de memoria.

`DATA dirección valor` requiere dos enteros. Se rechazan direcciones fuera de rango, duplicadas o superpuestas con las direcciones del código inicial. El cargador también rechaza programas vacíos, más de 100 instrucciones y ausencia de HALT.

La validación ocurre antes de `reset()`. Por ello, un archivo, texto o lista de instrucciones rechazados conservan el programa anterior, sus registros, memoria e historial. Las instrucciones cargadas se copian para que modificar después los objetos del llamador no altere el programa. Tener HALT es una condición del lenguaje, pero no demuestra que el flujo alcance esa instrucción.

Ciclo interno y errores

1. `step()` rechaza continuar si `error` está activo y devuelve sin ejecutar si ya hubo HALT.
2. Guarda PC, IR y ACC anteriores para la ejecución y su explicación.
3. Fetch lee Mem[PC], comprueba su tipo y copia la instrucción a IR.
4. Decode registra opcode y operando.
5. Prepara PC = PC + 1, después de Decode y antes de Execute.
6. `_execute()` modifica ACC, memoria, PC, dispositivos o detención.
7. Actualiza FLAGS y suma un ciclo completado.
8. Registra explicación, Fetch, Decode y Execute en la traza, y agrega el registro de la instrucción completada a `history`.

Un error de ejecución devuelve PC a la dirección fallida, conserva el diagnóstico en `error` y no suma un ciclo. Las ejecuciones siguientes generan `ExecutionFault` hasta reiniciar o cargar un programa válido. `halted` representa terminación por HALT; un fallo no se convierte en una terminación normal.

Si se cancela INPUT, se restauran PC e IR anteriores y se permite reintentar. ACC y los ciclos no cambian. Las tarjetas pueden indicar la cancelación, pero la máquina no queda bloqueada por un fallo permanente.

El motor, la consola y la GUI aplican el límite al total de ciclos completados desde la carga o reinicio, incluyendo pasos manuales. Reanudar no vuelve a conceder el límite completo. Alcanzar HALT en el último ciclo permitido termina correctamente. Sin programa cargado, `step()` informa la situación sin dejar un error permanente.

Cada elemento de `history` contiene ciclo, instrucción, PC y ACC antes/después, Z/N, cambios de memoria, fases y explicación. Solo se agregan instrucciones completadas; una cancelación o un fallo no inventa pasos, entradas o salidas. `outputs` e `input_values` recogen la salida y entrada confirmadas. `manual_edits` distingue cambios hechos por el usuario de las escrituras de STORE.

Operaciones de la arquitectura

Sintaxis	Efecto
`LOAD dir`	ACC = Mem[dir]
`STORE dir`	Mem[dir] = ACC
`ADD dir`	ACC = ACC + Mem[dir]
`SUB dir`	ACC = ACC - Mem[dir]
`JMP dir`	PC = dir
`JZ dir`	PC = dir si Z = 1
`JNZ dir`	PC = dir si Z = 0
`INPUT`	ACC = entero leído
`OUTPUT`	Emitir ACC
`HALT`	halted = True

Las direcciones válidas son 0 a 99. No se implementan etiquetas ni valores inmediatos en instrucciones. Los enteros utilizan la representación de Python y no un ancho de palabra fijo.

Interfaces y reinicio

`run_gui()` construye la ventana Tkinter y programa instrucciones con `after()`. Pausar cancela la próxima ejecución programada. Editor, memoria, tarjetas, traza e impresora se actualizan desde el mismo motor. La velocidad representa milisegundos entre instrucciones completas.

La GUI conserva el texto y los datos iniciales del programa cargado. Su botón Reiniciar vuelve a cargar esa fuente. Una edición de datos antes del primer paso pasa a formar parte de los datos que restaura; una edición posterior solo afecta a la ejecución actual. El código se modifica desde el editor, seguido de Cargar desde editor.

`run_cli()`, en `consola.py`, ofrece carga, paso, ejecución automática, registros, volcado, edición de datos y exportación. La opción Reiniciar llama a `restart()` y restaura el programa y sus datos iniciales, igual que el reinicio de GUI. `reset()` queda reservado al vaciado interno. El diagnóstico de ERROR se muestra también al consultar registros después de un fallo. EOF y Ctrl+C terminan el menú sin un traceback.

`main(argv=None)` valida los argumentos con argparse. Sin opciones abre la GUI; `--cli` abre el menú. El modo `--archivo RUTA` ejecuta sin interacción y recibe un `--entrada ENTERO` por cada INPUT. Si faltan valores, registra un error en lugar de esperar teclado. `--max-ciclos` controla el total y `--reporte RUTA` permite exportar incluso el estado de error. Devuelve 0 al completar correctamente y 1 ante fallo de carga, ejecución o guardado; argparse rechaza las combinaciones inválidas con código 2.

```
python simulador_vm.py --archivo examples/test6_entrada_salida.asm --entrada 7 --reporte
    output/reportes/entrada.html
```

Reportes de ejecución

`reportes.build_evidence()` crea una copia independiente del estado actual. El reporte no cambia si la VM continúa después. Incluye código, nombre, fecha, datos iniciales, registros, memoria completa, entrada, salida, historial, ediciones manuales, últimas fases y estadísticas de operaciones y saltos. Diferencia ERROR, SIN PROGRAMA, LISTA, PAUSADA y FINALIZADA; el estado exportado no equivale por sí mismo a una aprobación de la prueba.

`save_report()` selecciona formato por extensión. HTML es autónomo e imprimible, con textos escapados para que el programa ASM no se interprete como contenido activo. CSV utiliza separador de punto y coma, cabecera uniforme y UTF-8 con marca inicial para Excel; sus filas identifican programa, registros, pasos, escrituras, memoria, entradas, salidas y ediciones. Los textos que podrían convertirse en fórmulas se neutralizan. JSON conserva la instantánea estructurada completa. El guardado escribe primero un temporal y reemplaza el destino al completarse, evitando truncarlo ante una falla previa.

Evidencias y verificaciones

Desde la raíz del proyecto:

```
python -m tests.verificar_entrega
python -m unittest discover -s tests -v
python -m tests.verificar_gui
python scripts/generar_evidencias.py --incluir-gui
```

La verificación de GUI necesita un escritorio disponible. El generador sin `--incluir-gui` evita abrir ventanas e identifica qué verificaciones ejecutó. Los resultados reales quedan en `docs/EVIDENCIAS_PRUEBAS.md` y en `docs/evidencias/`; deben revisarse después de cada regeneración.

Las interfaces de GUI, consola y archivo utilizan el mismo módulo de reportes. La traza y `history` permiten seguir todas las instrucciones; el campo de últimas fases solo describe la última. El generador de evidencias ejecuta los ejemplos y las pruebas; el informe generado indica qué se comprobó y sus resultados sin fijar un conteo estático en este manual.

Generación del documento imprimible

```
python -m pip install -r scripts/requirements_documento.txt
python scripts/generar_documento.py
python -m pip install python-docx pypdfium2
python scripts/generar_word.py
python scripts/preparar_entrega.py
```

El PDF se guarda en `output/pdf/ENTREGA_GRUPO13.pdf` y el Word en `output/word/ENTREGA_GRUPO13.docx`. La guía para tres integrantes está en `output/exposicion/GUIA_TRES_INTEGRANTES.docx`; el PowerPoint está en `output/presentacion/DEFENSA_GRUPO13.pptx`. Las fuentes Markdown permanecen editables. Antes de entregar deben completarse los datos de portada y volver a generar los documentos.

El empaquetado produce `output/compartir/VM_STUDIO_GRUPO13.zip` con código y materiales seleccionados. Excluye cachés, temporales, material histórico y metadatos de Git; no incluye el ZIP dentro de sí mismo. Los compañeros deben descomprimir la carpeta completa para conservar las rutas de los ejemplos.

Recomendaciones de mantenimiento

- Cambiar el conjunto de instrucciones junto con el analizador, su ejecución, documentación y verificaciones pertinentes.
- Preservar la validación de carga antes de sustituir la memoria activa.
- Regenerar evidencias cuando cambie una semántica o un programa de ejemplo.
- Comprobar la GUI en el equipo de defensa, además de ejecutar las verificaciones automatizadas.
- Mantener el material histórico fuera de la ruta oficial de ejecución.

Anexo C. Evidencias de pruebas

Validación ejecutada: 2026-10-27T05:59:16-06:00 (America/Managua).

Este archivo se genera con resultados obtenidos al ejecutar el código actual. Los JSON conservan programas, entradas, registros, memoria, salida y fases de cada paso.

Reproducción

```
python scripts/generar_evidencias.py --incluir-gui
```

Sin `--incluir-gui` se ejecutan las pruebas de consola y se indica que la GUI no fue comprobada en esa ejecución.

Resultados funcionales

Prueba	Esperado	Obtenido	Estado
Suma	ACC=30; impresora=[30]; Mem[20]=30	ACC=30; PC=5; HALT=True; Mem[20]=30	CORRECTO
Resta	ACC=20; impresora=[20]; Mem[20]=20	ACC=20; PC=5; HALT=True; Mem[20]=20	CORRECTO
Almacenamiento	ACC=30; impresora=[30]; Mem[20]=30	ACC=30; PC=5; HALT=True; Mem[20]=30	CORRECTO
Condicional JZ	ACC=111; impresora=[111]	ACC=111; PC=9; HALT=True	CORRECTO
Bucle con JNZ	ACC=6; impresora=[6]; Mem[53]=6	ACC=6; PC=14; HALT=True; Mem[53]=6	CORRECTO
Entrada y salida	ACC=7; impresora=[7]; Mem[20]=7	ACC=7; PC=4; HALT=True; Mem[20]=7	CORRECTO

Los seis ejemplos se cargan desde su propio texto, con DATA cuando corresponde; no se agregan datos desde el catálogo de casos. Entrada del sexto programa: 7.

Traza numérica de suma

Paso	Instrucción	PC antes	PC después	ACC antes	ACC después	Mem[20]
1	LOAD 10	0	1	0	10	0
2	ADD 11	1	2	10	30	0
3	STORE 20	2	3	30	30	30
4	OUTPUT	3	4	30	30	30
5	HALT	4	5	30	30	30

Validaciones de carga

Situación	Programa	Estado
Instrucción inexistente	`MULT 10; HALT`	CORRECTO
Programa sin HALT	`LOAD 10; OUTPUT`	CORRECTO
Dirección superior inválida	`LOAD 100; HALT`	CORRECTO
Dirección negativa	`STORE -1; HALT`	CORRECTO
Error de sintaxis	`LOAD 10 20; HALT`	CORRECTO
Registro no válido	`LOAD R9; HALT`	CORRECTO
Registro usado como dirección	`LOAD ACC; HALT`	CORRECTO
DATA sobre código	`DATA 0 5; HALT`	CORRECTO
DATA duplicado	`DATA 90 1; DATA 90 2; HALT`	CORRECTO

Los mensajes completos y el tipo de excepción están en `docs/evidencias/errores_carga.json`.

Error de ejecución y estado de consola

El programa `LOAD 0; HALT` intenta leer código como un número. La VM conserva PC=0, registra la causa y exige reinicio. La consola informa ERROR.

```
==================================================
ESTADO DE LA MÁQUINA VIRTUAL (GRUPO 13)
==================================================
  PC  (Contador Programa)   : 00
  IR  (Registro Instrucción): LOAD 0
  ACC (Acumulador)          : 0
  FLAGS                     : [Z=1, N=0]
  ESTADO                    : ERROR
  CICLOS                    : 0
  CAUSA                     : La celda [0] contiene 'LOAD 0', no un dato numérico entero.
==================================================
```

Resultado: CORRECTO.

Verificador de entrega

Comando: `python -m tests.verificar_entrega`.

Estado: CORRECTO.

```
=== RESULTADOS DE CASOS DE PRUEBA ===
Suma: ACC=30, PC=5, IR=HALT, HALT=True, memoria={20: 30} -> OK
Resta: ACC=20, PC=5, IR=HALT, HALT=True, memoria={20: 20} -> OK
Almacenamiento: ACC=30, PC=5, IR=HALT, HALT=True, memoria={20: 30} -> OK
Condicional JZ: ACC=111, PC=9, IR=HALT, HALT=True -> OK
Bucle con JNZ: ACC=6, PC=14, IR=HALT, HALT=True, memoria={53: 6} -> OK
Entrada y salida: ACC=7, PC=4, IR=HALT, HALT=True, memoria={20: 7} -> OK

=== VALIDACIÓN DE ERRORES ===
Instrucción inexistente: OK
Programa sin HALT: OK
Dirección inválida: OK
Error de sintaxis: OK
```

```
Recarga de caso y restauración de datos: OK

TODAS LAS PRUEBAS PASARON.
```

Pruebas de regresión

Comando: `python -m unittest discover -s tests -v`.

Estado: CORRECTO.

```
test_archivo_con_entradas_repetidas_no_solicita_dialogos
    (test_consola.ConsoleTests.test_archivo_con_entradas_repetidas_no_solicita_dialogos) ... ok
test_argumentos_incompatibles_se_rechazan_antes_de_ejecutar
    (test_consola.ConsoleTests.test_argumentos_incompatibles_se_rechazan_antes_de_ejecutar) ... ok
test_bucle_batch_termina_y_exporta_error_en_limite_total
    (test_consola.ConsoleTests.test_bucle_batch_termina_y_exporta_error_en_limite_total) ... ok
test_carga_fallida_da_codigo_de_error_y_reporte_si_se_pidio
    (test_consola.ConsoleTests.test_carga_fallida_da_codigo_de_error_y_reporte_si_se_pidio) ... ok
test_falta_entrada_conserva_error_y_exporta_sin_bloquear
    (test_consola.ConsoleTests.test_falta_entrada_conserva_error_y_exporta_sin_bloquear) ... ok
test_menu_consulta_sin_carga_no_inutiliza_maquina
    (test_consola.ConsoleTests.test_menu_consulta_sin_carga_no_inutiliza_maquina) ... ok
test_menu_fin_de_archivo_y_ctrl_c_salen_sin_traceback
    (test_consola.ConsoleTests.test_menu_fin_de_archivo_y_ctrl_c_salen_sin_traceback) ... ok
test_menu_no_permite_editar_codigo_y_continua_utilizable
    (test_consola.ConsoleTests.test_menu_no_permite_editar_codigo_y_continua_utilizable) ... ok
test_menu_reinicia_datos_y_repite_programa
    (test_consola.ConsoleTests.test_menu_reinicia_datos_y_repite_programa) ... ok
test_archivo_con_bom_y_data_pasa_por_controlador_una_vez
    (test_regresiones.VMTests.test_archivo_con_bom_y_data_pasa_por_controlador_una_vez) ... ok
test_bucle_infinito (test_regresiones.VMTests.test_bucle_infinito) ... ok
test_cancelacion_y_error_no_producen_instrucciones_ficticias
    (test_regresiones.VMTests.test_cancelacion_y_error_no_producen_instrucciones_ficticias) ... ok
test_cancelar_y_reintentar (test_regresiones.VMTests.test_cancelar_y_reintentar) ... ok
test_carga_api_invalida_conserva_ejecucion_completa
    (test_regresiones.VMTests.test_carga_api_invalida_conserva_ejecucion_completa) ... ok
test_carga_invalida_preserva_programa (test_regresiones.VMTests.test_carga_invalida_preserva_programa) ... ok
test_cargador_copia_instrucciones_y_preserva_fuente
    (test_regresiones.VMTests.test_cargador_copia_instrucciones_y_preserva_fuente) ... ok
test_comentarios_y_limite_programa (test_regresiones.VMTests.test_comentarios_y_limite_programa) ... ok
test_consola_muestra_error_y_causa_hasta_recargar
    (test_regresiones.VMTests.test_consola_muestra_error_y_causa_hasta_recargar) ... ok
test_datos_declarados (test_regresiones.VMTests.test_datos_declarados) ... ok
test_diagnostico_registro_invalido_y_operando_no_entero
    (test_regresiones.VMTests.test_diagnostico_registro_invalido_y_operando_no_entero) ... ok
test_edicion_respeta_codigo_error_y_recuperacion
    (test_regresiones.VMTests.test_edicion_respeta_codigo_error_y_recuperacion) ... ok
test_entrada_tipo_invalido (test_regresiones.VMTests.test_entrada_tipo_invalido) ... ok
test_error_conserva_pc_y_bloquea (test_regresiones.VMTests.test_error_conserva_pc_y_bloquea) ... ok
test_explicaciones_reflejan_valores_y_decisiones
    (test_regresiones.VMTests.test_explicaciones_reflejan_valores_y_decisiones) ... ok
test_flags_iniciales_y_fases (test_regresiones.VMTests.test_flags_iniciales_y_fases) ... ok
test_halt_en_limite (test_regresiones.VMTests.test_halt_en_limite) ... ok
test_historial_registra_entrada_salida_y_cambio_de_memoria
    (test_regresiones.VMTests.test_historial_registra_entrada_salida_y_cambio_de_memoria) ... ok
test_limite_total_incluye_pasos_manuales (test_regresiones.VMTests.test_limite_total_incluye_pasos_manuales) ...
     ok
test_limites_y_fetch_invalido (test_regresiones.VMTests.test_limites_y_fetch_invalido) ... ok
test_memoria_rechaza_tipos_invalidos_sin_escribir
    (test_regresiones.VMTests.test_memoria_rechaza_tipos_invalidos_sin_escribir) ... ok
test_paso_sin_carga_no_bloquea_y_permite_cargar
    (test_regresiones.VMTests.test_paso_sin_carga_no_bloquea_y_permite_cargar) ... ok
test_reinicio_restaura_datos_editados_antes_del_primer_paso
    (test_regresiones.VMTests.test_reinicio_restaura_datos_editados_antes_del_primer_paso) ... ok
test_saltos (test_regresiones.VMTests.test_saltos) ... ok
test_csv_uniforme_formula_segura_y_numeros
    (test_reportes.ReportTests.test_csv_uniforme_formula_segura_y_numeros) ... ok
```

```
test_error_y_maquina_vacia (test_reportes.ReportTests.test_error_y_maquina_vacia) ... ok
test_guardado_roundtrip_y_extension_sin_destruir
    (test_reportes.ReportTests.test_guardado_roundtrip_y_extension_sin_destruir) ... ok
test_html_escapa_nombre_codigo_y_error (test_reportes.ReportTests.test_html_escapa_nombre_codigo_y_error) ... ok
test_salto_destino_secuencial_se_cuenta_por_condicion
    (test_reportes.ReportTests.test_salto_destino_secuencial_se_cuenta_por_condicion) ... ok
test_snapshot_independiente_y_estados (test_reportes.ReportTests.test_snapshot_independiente_y_estados) ... ok

----------------------------------------------------------------------
Ran 39 tests in 0.221s

OK
```

Interfaz gráfica

Comando: `python -m tests.verificar_gui`.

Estado: CORRECTO.

```
GUI OK: carga por controlador, DATA, pasos y pausa, reentrada protegida, editor pendiente, filtros, edición,
    límites, reinicio, errores y reportes JSON/HTML/CSV; ventana 1024x700 y 1366x768.
```

Archivos de evidencia

- `output/reportes/`: reportes HTML imprimibles, CSV para Excel y JSON completos de los seis casos; también se conserva el error de ejecución.
- `docs/evidencias/suma.json`, `resta.json` y `almacenamiento.json`.
- `docs/evidencias/condicional.json`, `bucle.json` y `entrada_salida.json`.
- `docs/evidencias/errores_carga.json` y `error_ejecucion.json`.
- `docs/evidencias/resumen.json`: fecha, resultados y comandos de esta validación.
- `docs/evidencias/verificador.txt` y `regresiones.txt`: salidas completas.
- `docs/evidencias/gui.txt`: resultado de GUI; indica si se omitió.

La prueba gráfica comprueba controles reales de Tkinter. La revisión visual del PDF y el ensayo oral de la defensa se realizan por separado.

Anexo D. Guía de defensa

Fecha de revisión: 26 de octubre de 2026.

Duración y distribución

La defensa se prepara para 15 a 20 minutos. El recorrido base dura aproximadamente 18 minutos. La versión de ensayo con guiones individuales está en `docs/exposicion/GUIA_TRES_INTEGRANTES.md` y `output/exposicion/GUIA_TRES_INTEGRANTES.docx`. Utilizar `output/presentacion/DEFENSA_GRUPO13.pptx` como apoyo visual y completar los nombres reales.

Tiempo	Responsable	Contenido y demostración
0:00 a 5:30	Integrante 1	Diapositivas 1 a 5: problema, objetivo, arquitectura, memoria y registros.
5:30 a 12:00	Integrante 2	Diapositivas 6 a 11: instrucciones, ciclo, suma y demostración, saltos y dispositivos.
12:00 a 18:00	Integrante 3	Diapositivas 12 a 16: errores, reportes, pruebas, organización y conclusiones.

Distribuir las intervenciones entre los integrantes reales antes de ensayar. Cada integrante debe poder explicar el programa, los saltos y el estado de error, aunque tenga asignada una sección distinta.

Preparación individual

Integrante 1: debe explicar por qué el proyecto permite observar la ejecución y distinguir simulación de procesador de sistema operativo completo. Mostrar `src/vm_studio/`: `motor.py` resuelve la arquitectura, `interfaz.py` presenta Tkinter, `consola.py` recibe comandos y `reportes.py` conserva evidencia. Evitar leer todo el código durante la exposición. Transición sugerida: «Con los componentes identificados, ahora veremos cómo un programa usa esas reglas».

Integrante 2: debe conocer las direcciones de la suma, los valores iniciales y cada cambio de PC, IR y ACC. Ejecutar cinco instrucciones con F7 y señalar la celda 20 después de STORE. Explicar que DATA no es una instrucción ejecutada, y que ADD 11 usa el contenido de la dirección 11. Después presenta JZ, el bucle JNZ y los controladores. Transición sugerida: «Ahora mi compañero mostrará cómo controlamos los errores y conservamos las evidencias».

Integrante 3: debe preparar un error de carga y un error de ejecución, explicar su recuperación y mostrar un reporte HTML. Relaciona el reporte con el estado observado, presenta las pruebas y la organización del código, y cierra con las limitaciones reales. No debe afirmar que las pruebas eliminan cualquier falla posible.

Preparación del equipo

Desde la raíz del proyecto, comprobar antes de la exposición:

```
python -m tests.verificar_entrega
python -m unittest discover -s tests -v
python -m tests.verificar_gui
python scripts/generar_evidencias.py --incluir-gui
python simulador_vm.py
```

Revisar el informe de resultados realmente obtenido en `docs/EVIDENCIAS_PRUEBAS.md`. Comprobar el tamaño de la ventana, la legibilidad y la disponibilidad de los archivos en la computadora que se utilizará. Dejar preparado

`output/pdf/ENTREGA_GRUPO13.pdf` y completar la portada con los datos reales.

Generar o conservar un reporte de respaldo:

```
python simulador_vm.py --archivo examples/test1_suma.asm --reporte output/reportes/suma.html
```

La aplicación y sus reportes no necesitan conexión. Abrir el PowerPoint y el reporte antes de empezar, y ensayar el cambio entre ventanas. Las pruebas deben ejecutarse antes de la exposición; durante ella basta con mostrar sus resultados y, si se solicita, ejecutar el verificador funcional.

Explicación de la arquitectura

La memoria tiene 100 celdas y contiene instrucciones y datos. PC indica dónde buscar la próxima instrucción. IR conserva la instrucción obtenida. ACC guarda los valores con los que se trabaja. La unidad de control coordina la búsqueda, interpretación y ejecución. Z indica si ACC es cero y N si es negativo.

El disco lee programas ASM, el teclado entrega enteros a INPUT y la impresora virtual muestra lo emitido por OUTPUT. La GUI y la consola utilizan el mismo motor.

El código está separado por responsabilidad. Los cambios visibles proceden de la misma instancia de `VirtualMachine`; no existen cálculos diferentes para consola y escritorio. Los reportes toman una instantánea del motor con sus pasos confirmados.

El proyecto conserva Python según autorización consignada por el grupo. La propuesta incluye referencias a Python y también indicaciones de Java. Si se consulta durante la defensa, presentar la constancia real que tenga el grupo; los archivos del proyecto no verifican por sí solos esa autorización.

Demostración de suma por pasos

Cargar `examples/test1_suma.asm` o el siguiente código completo:

```
DATA 10 10
DATA 11 20
LOAD 10
ADD 11
STORE 20
OUTPUT
HALT
```

Antes de empezar, mostrar PC = 0, ACC = 0 y Z = 1. Explicar que DATA instala datos sin ocupar posiciones de código. Las instrucciones ocupan las direcciones 0 a 4.

Paso	Explicación	Resultado visible
LOAD 10	Fetch obtiene LOAD 10; Decode reconoce una carga; Execute copia el dato 10.	PC = 1, IR = LOAD 10, ACC = 10.
ADD 11	Lee 20 de memoria y calcula 10 + 20.	PC = 2, ACC = 30.
STORE 20	Guarda 30 en la celda 20 sin cambiar ACC.	PC = 3, Mem[20] = 30.
OUTPUT	Envía el acumulador a la impresora virtual.	PC = 4, salida 30.
HALT	Finaliza la ejecución.	PC = 5, IR = HALT, cinco ciclos.

El PC aumenta después de Decode y antes de Execute. En un salto, Execute puede reemplazar ese valor. Las tarjetas muestran las fases de la última instrucción completa y la traza conserva las anteriores. F7 ejecuta toda la instrucción.

Saltos, bucle y dispositivos

Cargar `examples/test4_condicional.asm`. Explicar que 20 - 20 deja ACC en cero; JZ salta a la instrucción de dirección 6 e imprime 111. Las declaraciones y comentarios no cuentan para esa dirección.

Cargar `examples/test5_bucle.asm` y ejecutar automáticamente. Mostrar que el contador disminuye y que JNZ repite mientras no sea cero. El resultado es 3 + 2 + 1 = 6.

Cargar `examples/test6_entrada_salida.asm`, introducir 7 y comprobar memoria e impresora. Si hay tiempo, cancelar una solicitud de INPUT y reintentar para mostrar que no se cuenta una instrucción incompleta.

Error controlado

En el editor, intentar cargar:

```
MULT 10
HALT
```

El cargador debe rechazar MULT sin reemplazar el programa previo. Para distinguir un error durante la ejecución, utilizar después:

```
LOAD 0
HALT
```

El archivo es sintácticamente válido, pero la celda 0 contiene código. Al ejecutar LOAD, el motor informa que se intentó leer una instrucción como dato. Mostrar el diagnóstico, el PC conservado en 0 y la necesidad de reiniciar o cargar otro programa válido. En consola la consulta de registros también muestra ERROR y su causa.

Evidencias y resultados

Mostrar la tabla de resultados esperados y obtenidos en `docs/EVIDENCIAS_PRUEBAS.md`, una evidencia JSON y la traza numérica de la suma. Señalar el código, los datos iniciales, PC, IR, ACC, memoria, salida y diagnóstico si corresponde.

En el reporte HTML mostrar primero el estado y los registros; después una fila del historial donde STORE modifica Mem[20]. Explicar que CSV sirve para revisar registros en Excel y JSON conserva datos estructurados. Exportar durante una pausa o un error mantiene esa condición; no se presenta como una ejecución aprobada.

Para una presentación fluida, mostrar los archivos ya generados y ejecutar el verificador funcional por separado. El generador sustituye el informe con los resultados de esa ejecución; si se vuelve a generar sin `--incluir-gui`, esa ejecución no incluye las comprobaciones de ventanas.

Preguntas probables

¿Para qué sirve PC? Indica la dirección de la próxima instrucción. Los saltos permiten sustituir la continuación secuencial.

¿Para qué sirve IR? Conserva la instrucción obtenida en Fetch, incluso después de completarla, hasta que se obtenga otra.

¿Por qué se usa ACC? Simplifica la arquitectura: las cargas, sumas, restas, entradas y salidas trabajan sobre un registro principal.

¿Cuándo se activa Z? Después de una instrucción completada, si ACC es cero. Inicialmente también está activa porque ACC empieza en cero.

¿DATA es una instrucción de CPU? No. Es una directiva del cargador para establecer datos iniciales; no ocupa dirección de código ni consume ciclos.

¿Por qué LOAD ACC produce error? El acumulador es implícito. LOAD necesita la dirección de la celda que contiene el dato; por ejemplo, LOAD 10.

¿Tener HALT impide un bucle infinito? No. La carga exige HALT, pero un salto puede impedir alcanzarlo. El límite de ciclos detecta esa situación durante la ejecución.

¿Qué ocurre cuando se cancela INPUT? La instrucción queda pendiente, los registros anteriores se conservan y puede reintentarse.

¿Qué representa un estado ERROR? Una instrucción falló; el diagnóstico se conserva y no se permiten nuevos pasos hasta reiniciar o cargar un programa válido.

¿Qué significa reiniciar? El control de GUI y el menú de consola restauran el programa y sus datos iniciales con `restart()`. El método interno `reset()` es diferente: vacía el motor.

¿Cómo se calcula el límite de ciclos? Cuenta todas las instrucciones completadas desde carga o reinicio, incluidas las ejecutadas manualmente. Pausar o reanudar no devuelve el presupuesto completo. HALT en el último ciclo permitido es correcto.

¿Qué diferencia hay entre traza e historial? La traza contiene mensajes y diagnóstico. El historial contiene registros estructurados por instrucción completada, con PC y ACC antes/después, banderas y cambios de memoria. Los intentos cancelados o fallidos no cuentan como instrucciones completadas.

¿Qué ocurre si se acaba la entrada en modo por archivo? Se registra un error y el comando termina sin solicitar teclado. Se puede guardar ese estado con `--reporte`.

¿Qué limita el modelo? Tiene 100 celdas y diez instrucciones; no simula concurrencia, interrupciones, caché ni ancho de palabra fijo.

Cierre y recomendaciones

Concluir relacionando el programa con los objetivos: observar el ciclo, comprobar cambios de estado y reproducir resultados. Mencionar como ampliaciones posibles etiquetas simbólicas, nuevas operaciones y un modelo de palabra fija, dejando claro que no forman parte de esta versión.

Entregar el código oficial, seis casos completos, manuales, evidencias reales y el documento impreso con diagramas. Antes de la entrega definitiva, completar portada, revisar la versión PDF y conservar la constancia que tenga el grupo sobre la elección de Python.

Ensayo y respaldo

Hacer un ensayo completo con cronómetro y cambiar de integrante sin detener la demostración. Si el grupo supera 20 minutos, conservar la suma por pasos y resumir el bucle; no omitir la explicación de PC, IR, ACC o un error. Si falla una ejecución en vivo, leer el diagnóstico, reiniciar y repetir el ejemplo oficial. Si la computadora no permite abrir Tkinter, utilizar la consola y el reporte de respaldo, explicando qué se está mostrando.

Para compartir, utilizar `output/compartir/VM_STUDIO_GRUPO13.zip`. El compañero debe extraerlo y ejecutar desde la carpeta que contiene `simulador_vm.py`; copiar solo ese archivo omite los módulos y los programas de ejemplo.